

Aprendizaje Computacional

Dr. José Francisco Martínez Trinidad

Otoño 2026 - Bryan Ezequiel Violante Arriaga

Índice

1. Introducción al Curso	2
2. Aprendizaje	4
3. Clasificadores lineales y perceptrón	7
4. Convergencia y generalización (perceptrón)	10

1. Introducción al Curso

25 de agosto de 2026

Resumen rápido. Sesión inicial: contenido y dinámica del curso, temario, criterios de evaluación y el calendario del proyecto. La segunda mitad fue la introducción propiamente dicha: qué es aprender, por qué generalizar no es memorizar, de qué disciplinas se alimenta el área y la formalización de la función f que se quiere aprender contra la hipótesis h .

Info general

Contacto:

- Dr. José Francisco Martínez Trinidad
- Correo: fmartine@inaoep.mx

Temario:

1. Introducción y clasificadores lineales
2. Perceptrón
3. SVM
4. Clasificadores basados en distancia
5. ANNs
6. Evaluación, validación y overfitting
7. Métodos bayesianos
8. Árboles de decisión
9. Reglas de clasificación
10. Reglas de asociación
11. Selección de atributos
12. Clustering (K-means, gaussian mixture, jerárquicos)

Idea clave

No se trata de usar las herramientas como una caja negra. El objetivo es, como investigadores, saber cómo usarlas para generar nuevo conocimiento, y por eso hay que conocerlas a detalle: para poder innovar y proponer alternativas.

Evaluación

80 % proyecto:

- Equipo de dos personas
- Seleccionar un artículo reciente
- Proponer una solución alternativa
- Tres presentaciones

- Reporte final

20 % examen oral, a la entrega del proyecto, sobre un tema aleatorio del temario.

Calendario

- **25 de agosto.** Buscar un artículo publicado en 2026 que sea de interés, en una revista JCR.
- **3 de septiembre.** Entregar por correo una lista que indique cuál artículo seleccionó cada estudiante o equipo.
- **29 de septiembre o 1 de octubre.** Primera presentación:
 - Descripción del problema abordado en el artículo
 - Breve descripción del trabajo relacionado que menciona el artículo
 - Breve descripción de la solución propuesta en el artículo
 - Descripción de los experimentos que soportan dicha solución, incluyendo tablas y gráficas tomadas del artículo
 - Conclusiones a las que llegan los autores
 - Análisis de debilidades y posibles modificaciones para mejorar la propuesta de los autores
- **27 o 29 de octubre.** Segunda presentación: resumen del problema abordado.
- **24 o 26 de noviembre.** Presentación final: resumen del problema abordado y de la solución final.
- **30 de noviembre.** Entrega del reporte final. Se entrega en forma de artículo, con formato de <https://www.springer.com/>.

El reporte debe tener más o menos 12 páginas e incluir título, resumen, introducción, trabajo relacionado, solución propuesta, experimentos, conclusiones y trabajo futuro, y referencias.

Revistas JCR

- Expert Systems with Applications
- Knowledge-Based Systems
- Journal of Machine Learning Research

Buscador: <https://www.webofscience.com/wos/woscc/basic-research>

Recursos

- Weka 3
- scikit-learn

Ojito...

Hay que tratar de contactar al autor del artículo para que nos dé sus recursos y poder compararnos.

2. Aprendizaje

Que es el aprendizaje?

Es una disciplina dentro de las ciencias de la computación que surge como motivación de hacer que las computadoras realicen tareas complejas como las que hacen los humanos.

Aprender significa adquirir conocimiento, entendimiento o una habilidad de manera empírica o por estudio. La pregunta que queda abierta es cómo determinamos el éxito o el fracaso de la tarea.

Se busca que se logre un proceso de aprendizaje para que el sistema sea capaz de generalizar.

Idea clave

Generalizar no es memorizar. Aprender permite responder a problemas nuevos con el conocimiento adquirido.

Al inicio hubo varias tendencias. Una de ellas fue ver cómo aprenden los animales y de ahí trasladarlo a la máquina, con la premisa de que quizá el aprendizaje en un animal es diferente al de un ser humano, pero no resultó cierto.

Ojito...

No hay una demostración universal de lo que es el aprendizaje. Hay muchas definiciones y muchas escuelas.

Tipos de aprendizaje

- Supervisado
- No supervisado
- Por refuerzo

Tareas relacionadas con el aprendizaje

- Reconocimiento
- Diagnóstico
- Planeación
- Control de robots
- Predicción

Aprender puede ser percibir cambios en el comportamiento gracias a la experiencia. En nuestro contexto serían mejoras en actividades ya conocidas, y aprender tareas nuevas.

Hacer que las máquinas aprendan es muy útil, ya que nos simplifica la vida en muchos contextos: nos ayuda a hacer tareas complejas más rápido y a analizar datos masivos de una manera rápida.

Disciplinas relacionadas

- Estadística
- Lógica

- Matemáticas discretas
- Y muchas más

Todas las teorías que hay sobre cómo funciona el cerebro han ayudado bastante, ya que se trata de emular lo que se cree que ocurre en el cerebro humano cuando ocurre el aprendizaje, y trasladarlo a la computadora.

Otra disciplina que se vincula mucho con el ML es el control, la teoría de control. Igual los modelos psicológicos y los modelos evolutivos, entre muchos más.

Qué es lo que se aprende?

Diferentes estructuras computacionales: funciones, programas lógicos, máquinas de estado finito, gramáticas y sistemas para la resolución de problemas.

La idea es que tengo mi problema y tengo algunos ejemplos de él. Le doy a la computadora los ejemplos y la computadora me da como resultado el programa que me ayuda a generalizar cómo son esos ejemplos, de tal manera que cuando llegue un dato nuevo pueda clasificarlo, identificarlo y reconocerlo correctamente.

Más formalmente, se tiene una función f , que es la que queremos aprender, y otra función h , la hipótesis. Ambas reciben un vector de valores con N componentes. Queremos encontrar la función h cuya salida sea $h(X)$, y que sea capaz de reconocer un dato nuevo.

Supervisado y no supervisado

Hay dos tipos de aprendizaje, bueno tres, pero estos son los más comunes.

- **Supervisado.** Los datos de entrenamiento están etiquetados. A la función h , que es la que voy a aprender, le voy a dar un dato x , y esa función tiene que ser capaz de reconocer la nueva entrada.
- **No supervisado.** Los datos de entrenamiento no están etiquetados, y el algoritmo encuentra la relación entre los datos.

Vector de entrada

Al vector de entrada se le llama patrón, feature vector, instancia o ejemplo. Sus componentes son las variables o características, y esas variables pueden tomar valores reales, discretos o categóricos.

La salida puede ser un valor real, y entonces se trata de un predictor, o discreto, y entonces se trata de un clasificador.

Modos de entrenamiento

Hay varios modos de entrenamiento, dependiendo de los datos:

- Por batch
- De manera incremental
- En línea

Evaluación

Cuando ya tengo mi función h , ¿cómo la evalúo? ¿Cómo sé que está bien? Para eso se necesita un conjunto de prueba, que nos ayuda a ver qué tan bien la función h realiza su tarea, midiendo el error de h sobre ese conjunto. El conjunto de prueba es una partición del dataset original.

3. Clasificadores lineales y perceptrón

27 de agosto de 2026

Resumen rápido. Retomamos la idea de aprender una función que generalice y la aterrizamos en un ejemplo de control de acceso por rostro. De ahí salieron los clasificadores lineales, las funciones de pérdida y el perceptrón como algoritmo para ajustar los parámetros.

Recap

ML se trata de encontrar una función que fitee unos datos de entrenamiento para que sea capaz de generalizar. El proceso es:

1. Suponemos una función f que modela lo que queremos
2. Construimos hipótesis
3. Queremos construir la función h que emule a f
4. Entrenamos a h para que sea capaz de generalizar
5. Testeamos

Ejemplo: control de acceso por rostro

Supongamos que queremos construir un sistema que permita pasar a personas cuyos rostros sean conocidos para entrar al instituto. Tenemos los datos de las caras, y una imagen es un arreglo de bits.

Tenemos que construir una función

$$f : \mathbb{R}^d \rightarrow \{-1, 1\}$$

Es discreta porque es una tarea de clasificación. El clasificador tendrá un conjunto de vectores de entrenamiento con etiquetas, y estos vectores nos ayudarán a entrenar la función. Entonces queremos construir la función solo con los datos de entrenamiento, o sea con nuestra experiencia empírica.

Una manera sería ver un píxel de la imagen, de tal forma que al estar presente acepta a esa persona y la rechaza en otro caso, pero esto tiene varias limitaciones. ¿La función es útil para otras imágenes que no estén en el dataset? ¿Es capaz de generalizar?

Idea clave

Lo que se trata es de encontrar un modelo, una función que nos resuelva esa tarea de aprendizaje. No queremos funciones que fiteen perfectamente al conjunto de datos, ni súper generales, queremos algo en medio.

Clasificadores lineales

Lo que hacemos es definir una función lineal:

$$f(x; \theta) = \text{sign}(\theta_1 x_1 + \cdots + \theta_d x_d) = \text{sign}(\theta^\top x)$$

Se toma el signo para nuestro problema porque solo estamos considerando el caso binario: acepta o rechaza la entrada al edificio.

El modelo encontrará una línea, un plano que permita separar a las personas que pueden o no entrar al edificio. Ese plano es justo el lugar donde $\theta^\top x = 0$, y θ es el vector normal a él, así que ajustar θ es girar y mover el plano.

El chiste es encontrar la función a partir de las entradas que tengo en el entrenamiento. Cuando no es posible construir la función, queremos entonces descubrir otra que cometa el menor error posible.

Ojito...

En nuestro problema el clasificador lineal no guarda info de los píxeles vecinos, solo evaluaría el píxel individual.

Funciones de pérdida

Mencionamos que queremos encontrar el error. Esto se mide con la diferencia del dato esperado con el dato obtenido, y hay diferentes medidas de error.

A las funciones de error se les llama **loss**, función de error o función de pérdida, y hacen lo que mencionábamos: dicen qué tanta diferencia hay entre el dato observado y el esperado. Son esas funciones las que se tratan de optimizar.

El perceptrón

Queremos encontrar un clasificador lineal que minimice el error, o sea encontrar θ que minimice el error de entrenamiento:

$$E(\theta) = \frac{1}{n} \sum_{i=1}^n [1 - \delta(y^{(i)}, f(x^{(i)}; \theta))]$$

Donde δ vale 1 si sus dos argumentos coinciden y 0 si no, así que $1 - \delta$ vale 1 exactamente cuando el ejemplo quedó mal clasificado. Dicho de otra forma, $E(\theta)$ es la fracción de ejemplos de entrenamiento en los que el modelo se equivoca.

¿Pero cómo construir ahora esa función lineal? ¿Cómo ajustar los parámetros? ¿Hay alguna manera razonable de setear los parámetros de θ ?

Una opción es ajustar incrementalmente cada parámetro para que veamos que está bien. Un algoritmo simple de este tipo es el **perceptron update rule**:

$$\text{si } y^{(i)}(\theta^\top x^{(i)}) \leq 0 \quad \text{entonces} \quad \theta' = \theta + y^{(i)}x^{(i)}$$

En otras palabras, los parámetros son actualizados solo si hay error, y estos updates tienden a corregir errores. En nuestro caso, la clasificación de caras, cuando el modelo hace un error el signo no coincide con el de la etiqueta.

Supongamos que hacemos un error, y que los parámetros actualizados están dados por θ' . Si consideramos clasificar la misma imagen después del update:

$$\begin{aligned} y^{(i)}(\theta'^\top x^{(i)}) &= y^{(i)}((\theta + y^{(i)}x^{(i)})^\top x^{(i)}) \\ &= y^{(i)}(\theta^\top x^{(i)}) + (y^{(i)})^2 \|x^{(i)}\|^2 \\ &= y^{(i)}(\theta^\top x^{(i)}) + \|x^{(i)}\|^2 \end{aligned}$$

Como $y^{(i)} \in \{-1, 1\}$ su cuadrado siempre es 1, y $\|x^{(i)}\|^2 > 0$. O sea que el update le suma algo positivo, empujando ese ejemplo hacia el lado correcto del plano. Eso es lo que quiere decir que los updates tienden a corregir errores.

Ojito...

Esto da origen al reconocedor del perceptrón, y este reconocedor es útil para problemas lineales: solo puede separar con una línea o un plano, pero jamás podrá separar datos que no sean linealmente separables.

4. Convergencia y generalización (perceptrón)

1 de septiembre de 2026

Resumen rápido. En esta sesión vimos la convergencia del perceptrón, y la definición formal, que era el margen geométrico y cerramos con la intro a SVM

Punto de partida y supuestos

Seguimos con clasificadores lineales que pasan por el origen:

$$f(x; \theta) = \text{sign}(\theta^\top x),$$

donde θ denota los parámetros que tenemos que estimar en base a los ejemplos de entrenamiento, con ejemplos x y etiquetas y

Sea k el número de parámetros actualizados y $\theta^{(k)}$ el vector de parámetros después de k actualizaciones. Cada vez que el modelo se equivoca hace un update con la regla del perceptrón, y ese update es un k más

Para poder demostrar algo hacen falta dos supuestos:

1. **Las normas están acotadas:** existe R tal que $\|x^{(t)}\| \leq R$ para todo ejemplo.
2. **El clasificador lineal existe:** hay un θ^* , el θ ideal, que separa los datos con margen. Formalmente,

$$y^{(t)}(\theta^*)^\top x^{(t)} \geq \gamma > 0 \quad \text{para todo } t,$$

donde γ es el margen del clasificador, que es lo que nos dice que lo hace bien

Ojito...

En el ejemplo de las imágenes de las caras reconocidas para entrar al edificio, el segundo supuesto es justamente suponer que ese clasificador existe. Si los datos no son linealmente separables, no hay θ^* y nada de lo que sigue aplica

Convergencia en un número finito de updates

La convergencia se demuestra combinando dos resultados. El primero dice que el producto interno con el ideal crece linealmente, y el segundo que la norma cuadrada crece a lo más linealmente. Tomamos $\theta^{(0)} = 0$.

Parte 1: el producto interno crece linealmente

Supongamos que el update k se hizo sobre el ejemplo $(x^{(t)}, y^{(t)})$, o sea $\theta^{(k)} = \theta^{(k-1)} + y^{(t)} x^{(t)}$. Entonces

$$\begin{aligned} (\theta^*)^\top \theta^{(k)} &= (\theta^*)^\top \theta^{(k-1)} + y^{(t)} (\theta^*)^\top x^{(t)} \\ &\geq (\theta^*)^\top \theta^{(k-1)} + \gamma, \end{aligned}$$

donde la desigualdad es el supuesto del margen. Cada update le suma al menos γ , así que aplicándolo k veces desde $\theta^{(0)} = 0$:

$$(\theta^*)^\top \theta^{(k)} \geq k\gamma.$$

Parte 2: la norma cuadrada crece a lo más linealmente

Con el mismo update,

$$\begin{aligned}\|\theta^{(k)}\|^2 &= \|\theta^{(k-1)} + y^{(t)} x^{(t)}\|^2 \\ &= \|\theta^{(k-1)}\|^2 + 2 y^{(t)} (\theta^{(k-1)})^\top x^{(t)} + (y^{(t)})^2 \|x^{(t)}\|^2 \\ &\leq \|\theta^{(k-1)}\|^2 + R^2.\end{aligned}$$

El término de en medio se va porque el update ocurre precisamente cuando el ejemplo estaba mal clasificado, o sea cuando $y^{(t)} (\theta^{(k-1)})^\top x^{(t)} \leq 0$. Y el último se acota porque $(y^{(t)})^2 = 1$ y $\|x^{(t)}\| \leq R$. Repitiendo k veces:

$$\|\theta^{(k)}\|^2 \leq k R^2$$

Combinación: qué pasa con el ángulo

Si combinamos esos dos resultados podemos ver el coseno del ángulo entre el vector de la actualización k y el ideal:

$$\cos(\theta^*, \theta^{(k)}) = \frac{(\theta^*)^\top \theta^{(k)}}{\|\theta^*\| \|\theta^{(k)}\|} \geq \frac{k \gamma}{\|\theta^*\| \sqrt{k} R} = \sqrt{k} \frac{\gamma}{\|\theta^*\| R}.$$

El numerador se acotó por abajo con la parte 1 y el denominador por arriba con la parte 2. Como el coseno está acotado por 1:

$$\sqrt{k} \frac{\gamma}{\|\theta^*\| R} \leq 1 \quad \implies \quad k \leq \frac{R^2 \|\theta^*\|^2}{\gamma^2}.$$

Idea clave

Cada update acerca el coseno a 1 en una cantidad finita, o sea acerca el vector actual al ideal. Como el coseno no puede pasar de 1, solo se puede hacer un número finito de updates. Ahí está la convergencia

Ojito...

Está acotado, pero la cota depende del valor de los pesos: puede ser muy grande o muy pequeña, y de antemano no sabemos cuál.

Margen geométrico

El cociente $\|\theta^*\|^2 / \gamma^2$ nos dice qué tan difícil es el problema, o sea cuántos updates voy a tener que hacer. Su inverso es la distancia más chica que hay entre una instancia del conjunto de entrenamiento y mi región de decisión

Si esa distancia es grande hay bastante separación entre la superficie que divide las clases y las instancias, y es más fácil clasificar. Si es chica, es más difícil

Cómo se calcula

Se mide la distancia de la superficie de decisión a uno de los datos de entrenamiento. Sea $\tilde{x}$ el punto de la frontera más cercano a $x^{(t)}$, y ε esa distancia. Moverse de $x^{(t)}$ a la frontera es avanzar ε en la dirección normal, que es la de θ^* :

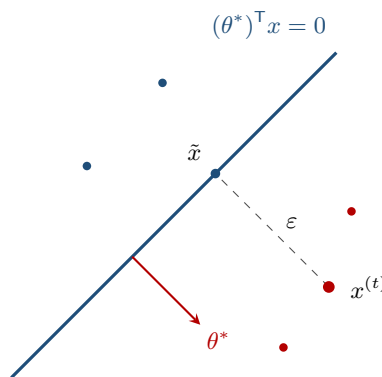
$$\tilde{x} = x^{(t)} - y^{(t)} \varepsilon \frac{\theta^*}{\|\theta^*\|}.$$

Evaluando el clasificador ideal en ese punto:

$$y^{(t)} (\theta^*)^\top \tilde{x} = \underbrace{y^{(t)} (\theta^*)^\top x^{(t)}}_{=\gamma} - \varepsilon \|\theta^*\| = \gamma - \varepsilon \|\theta^*\| = 0,$$

porque $\tilde{x}$ está sobre la frontera y ahí el clasificador vale cero. Despejando queda el **margen geométrico**:

$$\varepsilon = \frac{\gamma}{\|\theta^*\|}.$$



Por lo tanto el k , el número de veces que tengo que actualizar, se puede expresar en términos del margen geométrico:

$$k \leq \frac{R^2}{\varepsilon^2}.$$

Idea clave

Entre más ancho el pasillo que separa las clases, menos updates necesita el perceptrón. La dificultad del problema no está en cuántos datos hay ni en cuántas dimensiones, sino en qué tan apretado quedó ese pasillo

Generalización

Hasta ahorita asumimos que el conjunto de datos era linealmente separable, pero qué pasa si ahora no trabajo con mi conjunto de entrenamiento sino con datos que están fuera de él

Cuando llegue una instancia nueva, el clasificador lineal tendrá que ser capaz de clasificarla: se pone en el plano y se ve si la superficie separadora la separa correctamente

Idea clave

Lo que queremos ahora es un clasificador lineal que nos dé el máximo margen geométrico. De todas las superficies que separan bien el entrenamiento, la que deja el pasillo más ancho es la que tiene más holgura para los datos nuevos. Ese clasificador son las máquinas de vectores de soporte